# Sabaragamuwa University of Sri Lanka




# AWS PROJECT

| | |
|---|---|
| **Title** | AWS-Based Customer Churn Prediction and Analytics Solution |
| **Group No** | 06 |
| **Course Title** | AWS Programme |
| **Student Name** | PS Madushan, GDS Pasindi, HMUV Bandara, DMA Kanishka |
| **Index Number** | 22CDS0411, 22CDS0429, 22CDS0430, 22CDS0435 |

DEPARTMENT OF DATA SCIENCE
FACULTY OF COMPUTING
[15/10/2026]

# 1. Introduction

Customer churn occurs when a customer stops using a company's service. Understanding churn patterns helps a business identify customer groups that need better service and design retention strategies. This project analyses customer churn in a telecommunications dataset and demonstrates an AWS-based solution that stores data, validates and cleans it, analyses it with SQL, and serves churn predictions from a trained Random Forest model.

The project covers the following areas:

- AWS cloud storage and data processing
- Data validation and cleaning
- Exploratory Data Analysis (EDA)
- Machine learning model development and evaluation
- SQL analytics using Amazon Athena
- Business insights and recommendations
- A deployed prediction service with a batch (S3) interface and a real-time (API) interface
- GitHub collaboration and project documentation

# 2. Dataset

| Item | Detail |
|---|---|
| Dataset | Telco Customer Churn |
| Original file | `Telco-Customer-Churn.csv` |
| Size | 7,043 customer records |
| Target column | `Churn` |

The dataset contains customer demographics, contract type, tenure, monthly and total charges, payment method, services used, and churn status. The original CSV is uploaded to the Amazon S3 `raw/` folder and the processed dataset is saved to the S3 `processed/` folder. The team uses the agreed processed dataset for EDA, machine learning, and SQL analysis to keep results consistent.

# 3. Technologies Used

Python (pandas, NumPy, Matplotlib, Seaborn, scikit-learn), Google Colab, Amazon S3, AWS Lambda, Amazon API Gateway, Amazon CloudWatch, Amazon Athena, AWS IAM, and Git/GitHub.

# 4. AWS Architecture

The solution is deployed in the **eu-north-1 (Europe - Stockholm)** region. Figure 1 shows one prediction service that can be reached in two ways: a batch path through Amazon S3 and a real-time path through Amazon API Gateway.

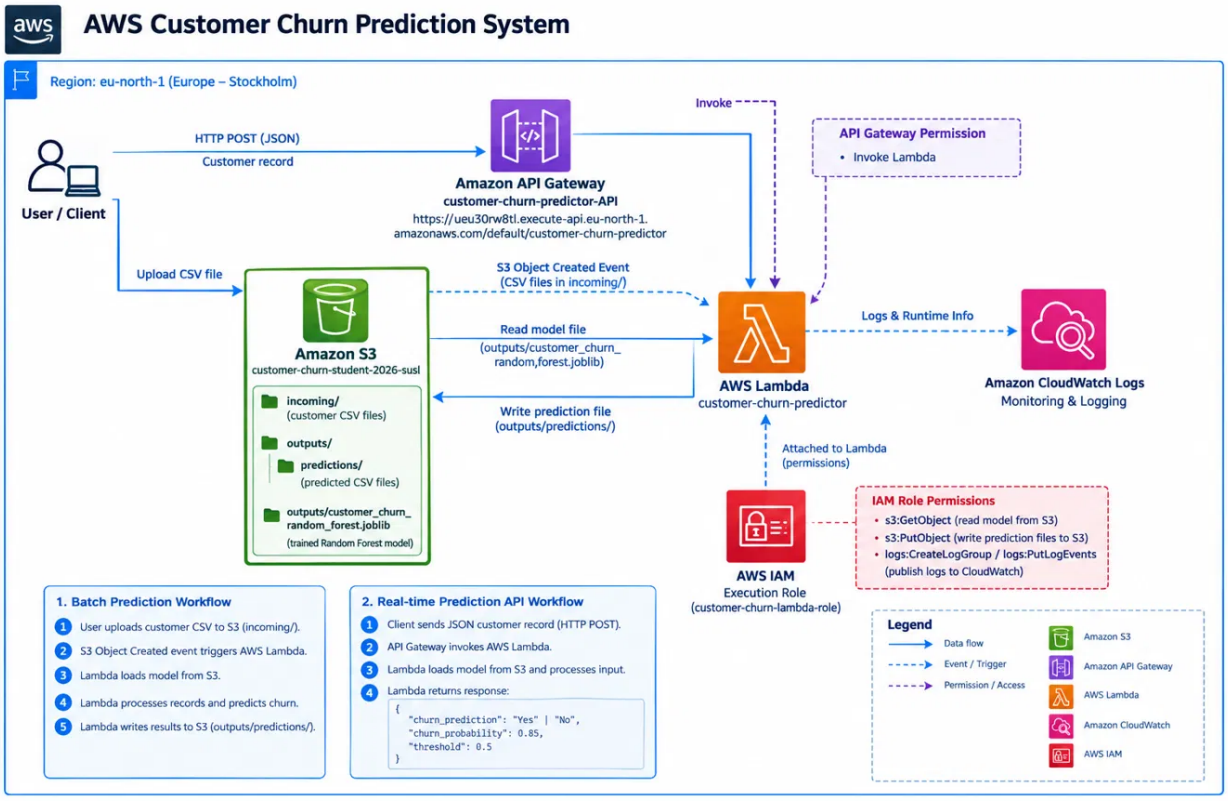



*Figure 1: AWS customer churn prediction system architecture*

## 4.1 AWS Resources

| AWS Resource | Name / Configuration |
|---|---|
| Region | `eu-north-1` |
| Amazon S3 bucket | `customer-churn-student-2026-susl` |
| AWS Lambda function | `customer-churn-predictor` |
| Amazon API Gateway | `customer-churn-predictor-API` |
| IAM execution role | `customer-churn-lambda-role` |
| Trained model | `outputs/customer_churn_random_forest.joblib` |
| Athena database / table | `customer_churn_db / customer_churn_db.churn_data` |

### 4.2 Components

**Amazon S3** holds the data and model using the following layout:

- `incoming/` - customer CSV files uploaded for batch prediction
- `outputs/predictions/` - predicted CSV files written by Lambda
- `outputs/customer_churn_random_forest.joblib` - the trained Random Forest model
- `raw/` and `processed/` - original and cleaned versions of the dataset (data-processing workflow, Section 5.1)

**AWS Lambda** (`customer-churn-predictor`) loads the model from S3, processes input records, predicts churn, and either writes a prediction CSV to S3 or returns a JSON response.

**Amazon API Gateway** exposes the Lambda function through an HTTP POST endpoint:

`https://ueu30rw8tl.execute-api.eu-north-1.amazonaws.com/default/customer-churn-predictor`

**Amazon CloudWatch Logs** receives logs and runtime information from Lambda for monitoring and troubleshooting.

**AWS IAM** provides the Lambda execution role. API Gateway is separately granted permission to invoke the Lambda function.

### 4.3 IAM Permissions

The Lambda execution role (`customer-churn-lambda-role`) is attached to the function and allows:

| Permission | Purpose |
|---|---|
| `s3:GetObject` | Read the model file and input files from S3 |
| `s3:PutObject` | Write prediction files to S3 |
| `logs:CreateLogGroup` / `logs:PutLogEvents` | Publish logs to CloudWatch |

The API Gateway permission allows it to invoke the Lambda function. Permissions should follow the principle of least privilege, and Lambda uses an execution role rather than hard-coded credentials.

## 5. Workflows

### 5.1 Data Processing Workflow (raw to processed)

1 The original CSV is uploaded to the S3 `raw/` folder.

2 An S3 Object Created event triggers the configured Lambda function.

3 Lambda reads the uploaded CSV.

4 The dataset is validated and cleaned.

5 The processed CSV is saved to the S3 `processed/` folder.

6 CloudWatch Logs are used to inspect execution messages and errors.

7 The processed dataset is used for EDA, machine learning, and SQL analysis.

This workflow uses different S3 prefixes from the prediction workflow below and is documented and verified separately. The S3 event notification should monitor only the intended input files so that Lambda writing output files does not cause repeated invocations.

## 5.2 Batch Prediction Workflow (S3)

1 The user uploads a customer CSV file to `incoming/`.

2 The S3 Object Created event (for CSV files in `incoming/`) triggers the `customer-churn-predictor` Lambda function.

3 Lambda loads the trained Random Forest model from S3.

4 Lambda processes the records and predicts churn.

5 Lambda writes the results to S3 under `outputs/predictions/`.

6 Execution logs are sent to CloudWatch.

The output filename is the input filename with `_predictions.csv` appended before the file extension.

- Example input: `incoming/customer_churn_test_data.csv`
- Example output: `outputs/predictions/customer_churn_test_data_predictions.csv`

The batch workflow was tested successfully and a prediction CSV was generated in the S3 output folder. Repeated uploads and fixed-filename output behaviour still need to be tested.

## 5.3 Real-Time Prediction Workflow (API)

1 A client sends customer details as an HTTP POST request with `Content-Type: application/json`.

2 API Gateway invokes the `customer-churn-predictor` Lambda function.

3 Lambda loads the model from S3 and processes the input.

4 Lambda returns the prediction as JSON.

The response contains three fields:

| Field | Meaning |
|---|---|
| `churn_prediction` | Predicted class ("Yes" or "No") |
| `churn_probability` | Estimated probability of churn |
| `threshold` | Classification threshold used by the prediction logic |

The endpoint was tested successfully using PowerShell. One example response from a test request is shown below.

```
{
  "churn_prediction": "No",
  "churn_probability": 0.4026,
  "threshold": 0.5
}
```

The model predicted "No" because the probability was below the classification threshold. The prediction is an estimate and does not guarantee a customer's future behaviour. The architecture diagram shows an illustrative response (probability 0.85) to explain the response format; it is not a test result.

## 6. Data Validation and Cleaning

Before analysis, the data is checked for:

- Required columns
- Missing values
- Duplicate records
- Incorrect data types
- Invalid numeric values
- Consistent column names and output formats

The Lambda cleaning process and notebook preprocessing should follow agreed rules, and any differences (missing-value handling, duplicate removal, data type conversion) are documented. Row counts and column names of the processed dataset are checked for consistency before other team members use it. The prediction Lambda also expects input fields to match the format required by the deployed model.

## 7. Exploratory Data Analysis

EDA is performed in `notebooks/01_eda.ipynb` on the agreed processed dataset, with charts saved to `outputs/figures/`. It covers:

1 Overall customer churn distribution

2 Churn by contract type

3 Monthly charges by churn status

4 Customer tenure by churn status

# 8. Machine Learning and Model Evaluation

Logistic Regression and Random Forest models were developed and evaluated on the same held-out test set of 1,409 customers (1,035 non-churn and 374 churn), using the training scripts `src/train_logistic_regression.py` and `src/train_random_forest.py`. The classification threshold for each model was selected during evaluation.

## 8.1 Summary of Results

| Model | Accuracy | Churn Precision | Churn Recall | Churn F1-score | Best CV F1-score | Selected Threshold |
|---|---|---|---|---|---|---|
| Logistic Regression | 0.7580 | 0.5313 | 0.7487 | 0.6215 | 0.6299 | 0.55 |
| Random Forest | 0.7395 | 0.5057 | 0.8262 | 0.6274 | 0.6325 | 0.40 |

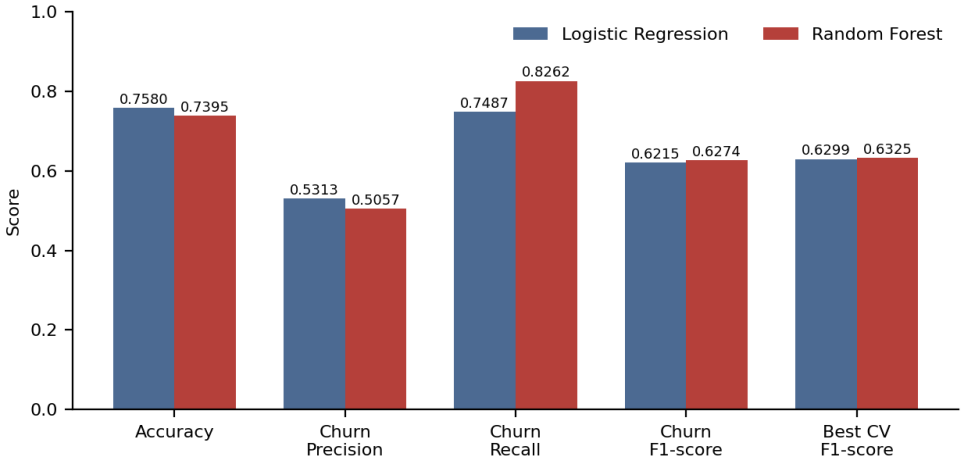



*Figure 2: Comparison of Logistic Regression and Random Forest on the test set*

## 8.2 Logistic Regression

Classification report (test set):

| Class | Precision | Recall | F1-score | Support |
|---|---|---|---|---|
| No churn | 0.89 | 0.76 | 0.82 | 1035 |
| Churn | 0.53 | 0.75 | 0.62 | 374 |
| Accuracy | | | 0.76 | 1409 |
| Macro avg | 0.71 | 0.76 | 0.72 | 1409 |
| Weighted avg | 0.80 | 0.76 | 0.77 | 1409 |

Confusion matrix (rows = actual, columns = predicted):

| | Predicted No | Predicted Yes |
|---|---|---|
| Actual No | 788 | 247 |
| Actual Yes | 94 | 280 |

The model correctly identified 280 of the 374 customers who churned (recall 0.7487) and correctly retained 788 of 1,035 non-churning customers.

## 8.3 Random Forest

Classification report (test set):

| Class | Precision | Recall | F1-score | Support |
|---|---|---|---|---|
| No churn | 0.92 | 0.71 | 0.80 | 1035 |
| Churn | 0.51 | 0.83 | 0.63 | 374 |
| Accuracy | | | 0.74 | 1409 |
| Macro avg | 0.71 | 0.77 | 0.71 | 1409 |
| Weighted avg | 0.81 | 0.74 | 0.75 | 1409 |

Confusion matrix (rows = actual, columns = predicted):

| | Predicted No | Predicted Yes |
|---|---|---|
| Actual No | 733 | 302 |
| Actual Yes | 65 | 309 |

The model correctly identified 309 of the 374 customers who churned (recall 0.8262). Important features include tenure, month-to-month contract, TotalCharges, and MonthlyCharges. Exact feature importance values are saved in `outputs/feature_importance.csv`.

## 8.4 Model Comparison

Logistic Regression achieved slightly higher accuracy (0.7580 against 0.7395) and churn precision (0.5313 against 0.5057). Random Forest achieved higher churn recall (0.8262 against 0.7487), a slightly higher churn F1-score (0.6274 against 0.6215), and a slightly higher best cross-validation F1-score (0.6325 against 0.6299). Random Forest therefore identified more of the customers who actually churned (309 against 280), at the cost of more false alarms (302 non-churning customers flagged against 247). When the goal is to find customers at risk of leaving so that retention actions can be targeted, higher recall is valuable, and the Random Forest model is the one deployed on AWS.

### 8.5 Model Deployment

The trained Random Forest model is stored in Amazon S3 and loaded by AWS Lambda for prediction. The deployed model expects the feature columns used in training, with the same preprocessing. Lambda derives a `TenureGroup` feature from `tenure` using these ranges:

- 0-12 Months
- 13-24 Months
- 25-48 Months
- 49-72 Months

The `customerID`, `Churn`, and `gender` columns are not input features for the deployed model. The threshold returned by the prediction service should match the threshold selected during model evaluation, and the deployed model must remain consistent with the model used in training.

## 9. SQL Analytics Using Amazon Athena

The analysis uses the Athena database `customer_churn_db` and table `customer_churn_db.churn_data`. Scripts are stored in `sql/` (`create_table.sql` and `analysis_queries.sql`), and the results of five queries are saved in `sql/query_results/`.

### 9.1 Overall Churn

| Churn status | Number of customers |
|---|---:|
| No | 5,174 |
| Yes | 1,869 |
| Total | 7,043 |

The overall churn rate is approximately 26.5%.

### 9.2 Churn by Contract Type

| Contract | Total customers | Churned customers | Churn rate |
|---|---:|---:|---:|
| Month-to-month | 3,875 | 1,655 | 42.71% |
| One year | 1,473 | 166 | 11.27% |
| Two year | 1,695 | 48 | 2.83% |

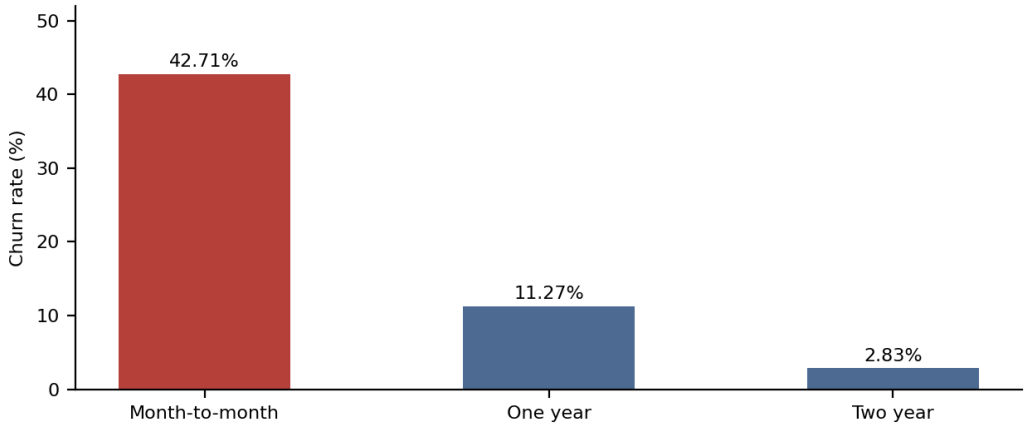



*Figure 3: Churn rate by contract type*

Month-to-month customers have the highest churn rate in this analysis.

## 9.3 Average Monthly Charges and Tenure

| Churn status | Average monthly charges | Average tenure (months) |
|---|---|---|
| No | 61.27 | 37.57 |
| Yes | 74.44 | 17.98 |

Customers who churned had higher average monthly charges and lower average tenure than customers who did not churn.

## 9.4 Churn by Payment Method

| Payment method | Total customers | Churned customers | Churn rate |
|---|---|---|---|
| Electronic check | 2,365 | 1,071 | 45.29% |
| Mailed check | 1,612 | 308 | 19.11% |
| Bank transfer (automatic) | 1,544 | 258 | 16.71% |
| Credit card (automatic) | 1,522 | 232 | 15.24% |

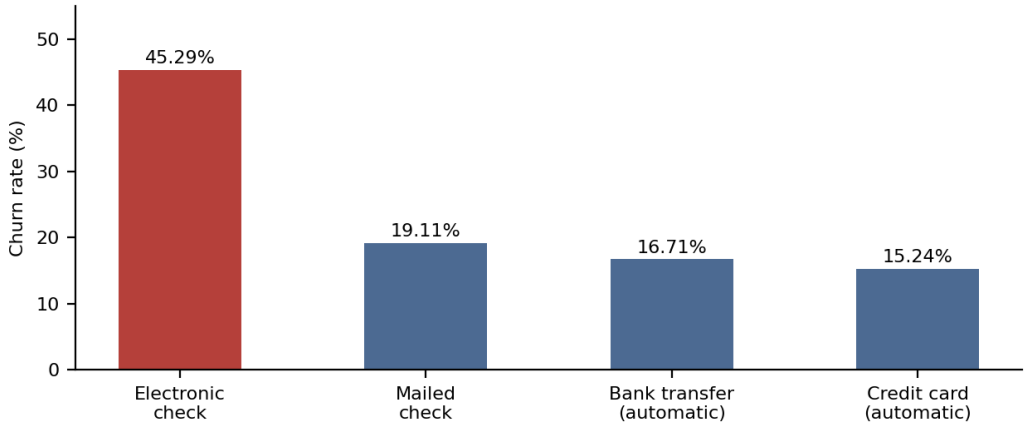



*Figure 4: Churn rate by payment method*

Electronic check users have the highest churn rate among the payment methods. These SQL results are descriptive: they show associations in the dataset and do not prove that a contract type, payment method, or charge level directly causes churn.

# 10. Business Recommendations

Based on the SQL analysis, the following actions are recommended.

## 10.1 Encourage Long-Term Contracts

Month-to-month customers have a churn rate of 42.71%, compared with 11.27% for one-year contracts and 2.83% for two-year contracts. **Recommendation:** test suitable discounts, benefits, or retention offers that encourage month-to-month customers to consider longer-term contracts, and monitor whether the offers improve retention.

## 10.2 Review the Payment Experience

Electronic check users have a churn rate of 45.29%, higher than the other payment methods. **Recommendation:** investigate whether customers experience difficulties with the electronic check process, explain convenient alternatives including automatic payments, and monitor whether retention improves.

## 10.3 Improve the Early Customer Experience

Customers who churned have an average tenure of 17.98 months, compared with 37.57 months for customers who did not churn, and their average monthly charges are also higher. **Recommendation:** provide early support to new customers, explain plan charges clearly, review whether customers are on suitable plans, and contact customers who show signs of dissatisfaction. Evaluate whether these actions reduce churn.

Further detail is available in `docs/business_recommendations.md`.

# 11. Testing and Verification

| Check | Status |
|---|---|
| S3 batch prediction workflow | Tested successfully |
| API prediction endpoint (PowerShell) | Tested successfully |
| Model evaluation results (both models) | Recorded from notebook output |
| Data validation and cleaning functions | To be confirmed |
| EDA notebook run end to end | To be confirmed |
| All five SQL queries executed | To be confirmed |
| Repeated uploads and fixed-filename output | To be tested |
| API error handling with invalid input | To be tested |
| CloudWatch Logs inspected | To be confirmed |

Checks are marked complete only after they have actually been performed. Screenshots and evidence are stored in `docs/demo_evidence/`.

# 12. Security and Cost Considerations

**Security**

- Never commit AWS access keys, secret keys, or passwords to GitHub, and review `.gitignore` before committing.
- Use an IAM execution role for Lambda instead of hard-coded credentials, with least-privilege permissions.
- Restrict access to S3 input files, the model, and prediction outputs to authorised users.
- Ensure API Gateway may invoke only the intended Lambda function.
- Avoid exposing sensitive customer information in public repositories or logs.

**Cost**

- Monitor usage and charges in the billing dashboard.
- Avoid unnecessary Lambda invocations, including recursive S3 triggers.
- Keep CloudWatch Logs and S3 storage within appropriate limits.
- Run Athena queries only when required and remove unused resources after testing.

## 13. Limitations

- Results depend on the quality and coverage of the dataset.
- The analysis shows associations, not proven causes of churn.
- Model performance depends on preprocessing, feature selection, and evaluation methods; precision for the churn class is around 0.5 for both models, so many flagged customers will not actually churn.
- The deployed model needs input in the expected format, and the churn probability is an estimate, not a guarantee.
- The batch pipeline must be tested with new and repeated uploads, and the API with invalid or incomplete input.
- Reported metrics must be reproducible from the saved code and agreed dataset.
- The raw-to-processed pipeline and the prediction pipeline use different S3 prefixes and must be documented and verified separately.

## 14. Team Contributions

| Member | Responsibility |
|---|---|
| Sandeepa | AWS Cloud and Data Pipeline |
| Pasindi | Data Cleaning and Exploratory Data Analysis |
| Avindu | Machine Learning and Model Evaluation |
| Uvindu | SQL Analytics, Business Insights, and Integration |

## 15. Conclusion

The project delivers an end-to-end AWS solution for customer churn: data stored and processed in Amazon S3, SQL analysis in Athena, and a Random Forest model served by AWS Lambda through both an S3-triggered batch workflow and an API Gateway endpoint, with CloudWatch monitoring and IAM-controlled access. Both Logistic Regression and Random Forest models reached about 74-76% accuracy, with Random Forest identifying more churning customers. The analysis highlights contract type, payment method, and tenure as key factors associated with churn, and the recommendations focus on retention offers, payment experience, and early customer support.